

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Staking Infrastructure	Documentation quality	High	
Timeline	2025-04-29 through 2025-05-07	Test quality	High	
Language	Solidity	Total Findings	9	Fixed: 5 Acknowledged: 4
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	docs-di.alluvial.finance ⓘ	Medium severity findings ⓘ	1	Fixed: 1
Source Code	https://github.com/AlluvialFinance/sms-lite ⓘ #d3ce772 ⓘ https://github.com/liquid-collective/tvs ⓘ #2c4095f ⓘ	Low severity findings ⓘ	4	Fixed: 3 Acknowledged: 1
Auditors	- Julio Aguilar Auditing Engineer - Leonardo Passos Senior Research Engineer - Jennifer Wu Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	4	Fixed: 1 Acknowledged: 3

Summary of Findings

Alluvial implements a modular Ethereum staking management system designed to support the validator changes introduced in the Pectra upgrade. The architecture centers around an upgradeable Orchestrator contract that manages transferrable validator sets (TVS), role assignments, and configuration updates. It employs a factory pattern for deployment, batch contracts for staking workflows, and configuration managers for allocation settings, using unstructured storage and both UUPS and Beacon proxy patterns to enable upgradeability.

Quantstamp audited the Alluvial codebase, which includes two repositories, one for the Orchestrator and one for the TVS, a test suite with high test coverage, and high-quality documentation. The audit identified a medium-severity issue related to the Staker's ability to redirect the Orchestrator's staked funds to other validators using consolidation, along with various lower-severity issues and suggestions aimed at improving overall robustness.

Fix-Review Update: The client has addressed all the issues by either fixing or acknowledging them. In addition, several minor enhancements were introduced to improve usability and simplify the codebase. Notably, a new feature was added that allows requesting validator exits during the TVS transfer process, enabling the receiving party to obtain the TVS in an exiting state.

ID	DESCRIPTION	SEVERITY	STATUS
ALL-1	Staker Can Steal TVS Validator Funds	• Medium ⓘ	Fixed
ALL-2	TVS Transfer Protection Can Be Bypassed with Custom Beacon Implementation	• Low ⓘ	Acknowledged
ALL-3	Orchestrator Cannot Upgrade TVS Beacon Implementation	• Low ⓘ	Fixed
ALL-4	Withdrawal and Consolidation Functions Will Fail with Multiple Selections	• Low ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
ALL-5	The Orchestrator Is only Upgradeable when It Is Not Paused	• Low ⓘ	Fixed
ALL-6	Orchestrator Ownership Transfer Does Not Revoke Previous Owner's Granted Roles	• Informational ⓘ	Acknowledged
ALL-7	Staker Can Redirect Validator Funds if TVS Beneficiary Is Set to Orchestrator	• Informational ⓘ	Fixed
ALL-8	Limited Sweep Functionality when Orchestrator Is TVS Owner	• Informational ⓘ	Acknowledged
ALL-9	Consolidations May Trigger Switch to Compounding Withdrawals Credential	• Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: [https://github.com/AlluvialFinance/sms-lite\(d3ce7720e4c07bd20859712537da0b1f9c3cdeff\)](https://github.com/AlluvialFinance/sms-lite(d3ce7720e4c07bd20859712537da0b1f9c3cdeff))
Files: src/*

Repo: [https://github.com/liquid-collective/tvs\(2c4095f148388a42ce8635e2998d222bbf1f1e45\)](https://github.com/liquid-collective/tvs(2c4095f148388a42ce8635e2998d222bbf1f1e45))
Files: src/*

Files Excluded

Repo: [https://github.com/AlluvialFinance/sms-lite\(d3ce7720e4c07bd20859712537da0b1f9c3cdeff\)](https://github.com/AlluvialFinance/sms-lite(d3ce7720e4c07bd20859712537da0b1f9c3cdeff))
Files: none

Repo: [https://github.com/liquid-collective/tvs\(2c4095f148388a42ce8635e2998d222bbf1f1e45\)](https://github.com/liquid-collective/tvs(2c4095f148388a42ce8635e2998d222bbf1f1e45))
Files: none

Operational Considerations

The system relies on centralized roles across both the Orchestrator (SMS) and TVS components. The `DEFAULT_ADMIN_ROLE` in the Orchestrator has extensive privileges, including managing upgrades, pausing the system, and assigning or revoking other roles. The owner of the TVS contracts can sweep funds, change beneficiaries, initiate withdrawals, consolidate validators, and upgrade implementations in upgradeable variants. Additionally, there are centralized actors within the Orchestrator, such as the Keeper and Staker, that have delegated power from the owner as mentioned in the Key Actors and Capabilities section. These powers introduce a high level of trust in the admin key. It is assumed that mitigation relies on multisig wallets, and strict key management policies.

The Orchestrator uses the UUPS proxy pattern for upgrades, while TVS contracts use beacon proxies. In both cases, upgrades are authorized by privileged roles. Even though upgradeability offers flexibility, it also introduces risk if upgrades are deployed without proper testing or review. To mitigate this, the system should rely on robust off-chain processes for implementation review, and thorough testing.

The `TVSFlexibleImmutable` contract permits arbitrary call execution, including delegate calls. While this design enables operational flexibility, it also increases the risk of unintended or unauthorized state changes. Calls made through this mechanism should be carefully reviewed to prevent misuse.

The system leverages new validator rules introduced in the Pectra update, including the ability to consolidate multiple source validators into a single target validator. When consolidation is initiated from the TVS, funds are transferred to the target validator’s withdrawal credential. This mechanism introduces timing risks. When transferring ownership of an Orchestrator, careful timing considerations must be factored in to prevent underlying TVS value staleness issues. If the Orchestrator owner initiates consolidation of the TVS to another validator before the Orchestrator transfer, the underlying TVS validator becomes effectively exited, potentially resulting in the new owner receiving an empty or devalued underlying TVS. Similarly, if the Orchestrator owner initiates a validator withdrawal exit prior to transfer, the funds are sent to the TVS but may be swept to the previous owner's beneficiary before the Orchestrator ownership is transferred to the new owner. This timing vulnerability creates a window where value extraction can occur, leaving the new owner with an Orchestrator with underlying TVSs that do not reflect their expected value. Although there is a trust assumption on Orchestrator owners, they may be unaware of these issues and make unintended mistakes. We strongly recommend educating owners on the risks of Orchestrator transfer and underlying TVS value staleness.

Finally, while the beacon contract used by the Orchestrator to create TVSs is out of scope, its presence and control have direct security implications for the SMS system. Since the beacon governs the implementation logic for upgradeable TVSs, its owner should be treated with the same level of caution as other privileged roles in the system.

Key Actors And Their Capabilities

Orchestrator

- 1. **DEFAULT_ADMIN_ROLE:**
 - Can grant and revoke roles
 - Can unpause the contract
 - Can transfer the orchestrator
 - Can set the config manager address
 - Can authorize upgrades
 - Cannot be renounced or transferred via standard role management functions
- 2. **PAUSER_ROLE:**
 - Can pause the contract
- 3. **DEPOSITOR_ROLE:**
 - Can deposit ETH into the contract
- 4. **VALIDATOR_MANAGER_ROLE:**
 - Can request exits
 - Can consolidate validators
- 5. **ETH_WITHDRAWER_ROLE:**
 - Can withdraw ETH from the contract
- 6. **DEPOSIT_CONSENSUS_LAYER_ROLE:**
 - Can deposit validator sets to the consensus layer
- 7. **VALIDATOR_SET_TRANSFER_ROLE:**
 - Can transfer validator sets
- 8. **VALIDATOR_SET_CREATOR_ROLE:**
 - Can create validator sets
- 9. **CONFIG_MANAGER_ROLE:**
 - Can configure system parameters like min sweep amount, allocation config, etc.

10. **BENEFICIARY_ROLE**:
 - Role related to receiving funds
11. **ORCHESTRATOR_DEPLOYER_ROLE** (in OrchestratorFactory):
 - Can deploy new orchestrator instances
- The system also has predefined role configurations for specific actors:
- **Owner**: Has all roles (`DEFAULT_ADMIN_ROLE` and others).
 - **Staker**: Has roles `DEPOSITOR_ROLE` , `VALIDATOR_MANAGER_ROLE` , `VALIDATOR_SET_TRANSFER_ROLE` , `PAUSER_ROLE` and `CONFIG_MANAGER_ROLE` .
 - **Keeper**: Has roles `VALIDATOR_SET_CREATOR_ROLE` and `DEPOSIT_CONSENSUS_LAYER_ROLE` .

TVS

1. Owner
 - Set by the `_setupSecurity` function during initialization.
 - Can transfer ownership to a new owner (but cannot renounce ownership).
 - Can set a new beneficiary address.
 - Can sweep funds to any address.
 - Can initiate withdrawals and consolidations.
 - In `TVSUpgradeable` : Can update the beacon address.
 - In `TVSFlexibleImmutable` : Can execute arbitrary calls and batches of calls.
2. Beneficiary
 - Receives ETH when swept from the TVS contract.
 - Can be updated by the owner.
 - Can be a contract that implements `ITVSSweepBeneficiary` interface.

Findings

ALL-1 Staker Can Steal TVS Validator Funds

• Medium ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `4884955d088a610643b5bcd95d3de2f7ea231a1` .
The client provided the following explanation:

Great find! We appreciate the feedback and have removed consolidations from the Orchestrator.

i Update

The consolidate function has been removed from the Orchestrator to prevent unauthorized fund transfers. Therefore, it is not possible to consolidate through the Orchestrator. If the Orchestrator wants to initiate TVS consolidation, it must transfer the TVS to a non-orchestrator.

File(s) affected: `sms-lite/src/batchContracts/BatchPectra.sol` , `tvsrc/TVS.sol`

Description: The `consolidate` function enables merging a source validator's stake into a target validator through EIP-7251 (Pectra upgrades). In the current architecture, the Transferable Validator Set (TVS) contract serves as the withdrawal credential for validators, with all funds directed to this address. The Orchestrator smart contract, as the TVS owner, delegates certain permissions to addresses with the Staker role.

The vulnerability exists because when a Staker invokes `consolidate` through the Orchestrator, they can specify any target validator to receive the source validator's funds. Since the consolidation request originates from the TVS (the withdrawal credential on the execution layer), it satisfies the EIP-7251 requirements, and the request is processed on the Consensus Layer.

Unlike withdrawals, which must return funds to the TVS withdrawal credential address, the consolidation feature allows unrestricted selection of target validators. This means a compromised or malicious Staker could redirect funds to validators under their control, effectively stealing staked assets while appearing to perform legitimate consolidation operations.

Recommendation: Restrict access to the `consolidate` function so that it can only be called by the Orchestrator owner.

ALL-2
TVS Transfer Protection Can Be Bypassed with Custom Beacon Implementation

• Low ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

We acknowledge the issue. However, we do not plan to implement a fix at this time. The described attack vector requires a malicious contract owner, and our system operates under the assumption that all owners are trusted and have undergone KYC verification. Given this trust model, we consider the risk to be acceptable.

File(s) affected: `tvsrc/TVSUpgradeable.sol`

Description: The `TVSUpgradeable` contract includes a `transfer` function designed to safely transfer ownership of a TVS while preventing the previous owner from retaining control. This function is intended to set the beacon to an immutable implementation before transferring ownership:

```
function transfer(address newBeneficiary, address newOwner) external override virtual onlyOwner {
    _setBeacon(immutableBeacon);
    _transfer(newBeneficiary, newOwner);
}
```

However, this protection mechanism can be circumvented. If an owner updates the beacon to point to a custom implementation that overrides the `transfer` function with a version that skips setting the beacon back to `immutableBeacon`, the security measure becomes ineffective.

This allows the previous owner to maintain control over the TVS even after transferring ownership. There is no guarantee that the original implementation will be reset upon transfer to the new owner with the current upgrade beacon design.

Recommendation: To provide better protection for new owners during transfers, consider implementing a whitelist of approved beacon implementations. This would ensure that only verified and secure implementations can be used, preventing malicious modifications to the transfer function that could compromise the new owner's control.

ALL-3 Orchestrator Cannot Upgrade TVS Beacon Implementation

LowFixed

Update

Marked as "Fixed" by the client.
Addressed in: `6663cac07b589cc1e8314aad4a37439690334fd1`.
The client provided the following explanation:

We have added a fix for this finding, such that the TVS beacon implementation can be upgraded from within the Orchestrator.

File(s) affected: `sms-lite/src/OrchestratorBase.sol`, `tvsrc/TVSUpgradeable.sol`

Description: The Transferable Validator Set (TVS) is deployed as an upgradeable contract (`TVSUpgradeable`) with the Orchestrator as its owner. The `TVSUpgradeable` contract includes a `setBeacon` function that allows the owner to upgrade the beacon implementation. However, there is currently no implemented function in the Orchestrator that would allow for updating the TVS beacon implementation. Since the Orchestrator is the owner of the TVS, and only the owner of the TVS can call the `setBeacon` function, this creates a functional limitation. This limitation means that despite the TVS being deployed as an upgradeable contract, its beacon implementation cannot be upgraded through the current contract architecture.

Recommendation: Implement a dedicated function in the Orchestrator contract that allows the Orchestrator owner to invoke the TVS `setBeacon` function.

ALL-4Withdrawal and Consolidation Functions Will Fail with Multiple Selections

LowFixed

Update

Marked as "Fixed" by the client.
Addressed in: `044d09b4c677c8a11d3b5e7c79036ad64ff4b210`.

File(s) affected: `sms-lite/src/batchContracts/BatchPectra.sol`

Description: The `_withdrawFromValidatorSets` and `_consolidateValidators` functions include loop structures suggesting they can process multiple selections. However, they will fail if the selection array length is greater than 1 because the entire `msg.value` is forwarded in the first iteration. The test suite does not identify this issue because the testsuite only considers selection arrays of length 1.

Recommendation: We recommend updating the function to properly handle the required fees for each selection. Enhance the test suite to include tests with selection lengths greater than 1.

ALL-5 The Orchestrator Is only Upgradeable when It Is Not Paused

Low Fixed



Update

Marked as "Fixed" by the client.
Addressed in: 9b17f2706299b89131c243749d83cae5d4417623 .

File(s) affected: src/OrchestratorUpgradeableBase.sol

Description: The OrchestratorUpgradeableBase contract overrides the function _authorizeUpgrade() and adds the isNotPaused modifier. However, in case of an update for security reasons, it might be desired to pause the contract first to mitigate the impact. The current implementation would not allow that.

Recommendation: We recommend removing the isNotPaused modifier.

ALL-6 Orchestrator Ownership Transfer Does Not Revoke Previous Owner's Granted Roles

Informational Acknowledged



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Our current implementation assumes that an Orchestrator transfer occurs within the same custodian and ownership domain. As such, the mechanism is intentionally designed to update the owner only without modifying or revoking roles previously granted to other users. The "transfer Orchestrator" function does not transfer the Orchestrator to a new entity, but simply updates its owner.

To eliminate future ambiguity, we've renamed the function to changeOwner (75703b3d9585876b3cebfc9ebb4f6d2f59c7586a). However, for the reasons outlined above, we are not implementing the recommendation as suggested.



Update

This issue was initially rated as High severity due to the risk of lingering role-based permissions after an Orchestrator ownership transfer. However, after clarification from the client that such transfers are intended to occur within the same custodian or operational domain and do not imply a change in controlling entity, we agree that the current behavior aligns with the system's design. Based on this, we have downgraded the severity to Informational. The client renamed the function changeOwner() to reflect this intent more clearly. It is important to note that role assignments are NOT automatically reset during ownership changes; any roles that should not persist must be explicitly revoked by the new owner. We recommend clarifying this intention in the documentation or code comments to avoid future misunderstandings.

File(s) affected: sms-lite/src/OrchestratorBase.sol

Description: The transferOrchestrator function transfers ownership of the Orchestrator contract to a new address. While the function grants owner roles to the new address and revokes these roles from the previous owner through _revokeRoles(msg.sender, _getOwnerRoles()), it does not revoke any addresses that might have been assigned roles by the previous owner (e.g. Staker or Keeper).

This oversight could potentially lead to unauthorized access and control over protocol functions after ownership transitions.

Recommendation: Modify the _transferOrchestrator() function to revoke all roles granted by the previous owner as part of the ownership transfer process. Unless all the addresses are tracked on-chain, it is not straightforward to revoke such addresses. This may require a design change.

ALL-7 Staker Can Redirect Validator Funds if TVS Beneficiary Is Set to Orchestrator

Informational Fixed



Update

Marked as "Fixed" by the client.
Addressed in: c0751a25fd4210eaf215d1ca49fa6d9cfd0b3f1 .
The client provided the following explanation:

Our program architecture assumes that the Orchestrator will never be the TVS beneficiary, so the described attack should not be a valid concern. However, we acknowledge the lack of separation of concerns. To address this, we've removed the `ETH_WITHDRAWER_ROLE` from the staker's permissions and updated the NatSpec to clearly state that the beneficiary address must not be set to the Orchestrator address in this version of SMS (<https://github.com/AlluvialFinance/sms-lite/commit/6b3ad9d7f3c1dbdf2060e6f32b1e2108b463e27e>).

Update

This issue was originally of Medium severity as the combination of permissions granted to the Staker role could enable full control over validator funds if the TVS beneficiary were set to the Orchestrator contract. Following clarification from the client that their architecture assumes the Orchestrator will never be the beneficiary, and that this configuration is not supported in the current version, we agree that the scenario falls outside the intended use case. The client has since removed the `ETH_WITHDRAWER_ROLE` from the Staker and will add documentation explicitly stating that the Orchestrator should not be set as the TVS beneficiary. We have therefore downgraded the issue to Informational. Please note that ETH withdrawal from the Orchestrator is now restricted to the owner only, and granting this ability back to Stakers could reintroduce a potential unauthorized access path. To further reduce misconfiguration risk, we recommend enforcing the assumption that the Orchestrator will never be the TVS beneficiary directly in the code.

File(s) affected: `sms-lite/src/batchContracts/BatchSweep.sol`, `sms-lite/src/batchContracts/BatchPectra.sol`, `sms-lite/src/OrchestratorBase.sol`, `tvsrc/TVS.sol`

Description: Based on discussions with the client, the `withdrawETH` function in the Orchestrator contract was designed for scenarios where the Orchestrator is set as the beneficiary of the TVS (Transferable Validator Set). However, the Staker role, configured in `_configureStaker`, has been granted access to this function and other functions, creating a potentially problematic permission combination below.

The Staker role already has privileges to:

1. Invoke the `sweepToBeneficiary` function, which can transfer any TVS balance to the TVS beneficiary (non-permissioned),
2. Call `requestExits` to trigger validator exits via exit daemons off-chain.
3. Call `withdrawFromValidatorSets` to trigger [validator exit process](#) via execution layer post Pectra upgrade.

If the TVS beneficiary is set to the Orchestrator contract address (a future configuration option planned by the client), these permissions create a withdrawal flow controlled by the Staker from the Orchestrator validators:

- The Staker can request validator exits through `requestExits` (off-chain) or `withdrawFromValidatorSets` (on-chain)
- After exits are processed on the Consensus Layer, funds are sent to the TVS (validator withdrawal credential)
- The Staker can then sweep these funds to the TVS beneficiary (the Orchestrator)
- Finally, the Staker can call `withdrawETH` on the Orchestrator to redirect these funds to any destination of their choosing

This permission configuration effectively gives the Staker role complete control over validator funds in this specific scenario, which may exceed intended access control boundaries despite being technically supported by the system design.

Recommendation: Consider delineating the Staker role further to prevent this control of funds. Only the Orchestrator owner should have access to the validator funds. Alternatively, document this risk if the TVS beneficiary is set to the Orchestrator contract address.

ALL-8 Limited Sweep Functionality when Orchestrator Is TVS Owner

• **Informational** ⓘ **Acknowledged**

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Whilst we acknowledge that this functionality could be beneficial in the future, we are happy to exclude custom sweep destinations from this release.

File(s) affected: `sms-lite/src/OrchestratorBase.sol`, `tvsrc/TVSUpgradeable.sol`

Description: In the current implementation of the `BatchSweep` contract, the `_sweepToBeneficiary` function only allows sweeping funds to the beneficiary address stored in the TVS when called with `address(0)` as the beneficiary parameter. This creates a limitation where even the Orchestrator owner can only sweep ETH to the Orchestrator itself via this function.

The TVS contract's `_sweep` function includes a check that requires the caller to be the owner when specifying a custom beneficiary other than `address(0)`. However, the `BatchSweep` contract's `_sweepToBeneficiary` function does not provide any way to specify a custom beneficiary, as it always passes `address(0)` to the TVS's `sweep` function. This means that while the TVS contract supports sweeping to custom destinations for the owner, this functionality is not exposed through the `BatchSweep` interface, creating an unnecessary restriction on the Orchestrator owner's capabilities.

Recommendation: Implement a new function in the `BatchSweep` contract that allows the Orchestrator owner to sweep TVS funds to custom destinations. This function should be restricted to the owner and should accept a destination parameter.

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

This implementation purposely follows the Pectra EIP-7251 implementation, where if source and target validator are the same for a consolidation request, the validator will be upgraded to 0x02 withdrawal credentials.

We do not see an issue with this and will refrain from making any changes.

File(s) affected: lib/tvs/src/components/TVS.sol

Description: The `consolidate` function in the TVS implementation, when iterating over the consolidation requests, submits them to Ethereum's system contract without checking if the source validator pubkey is different from the target validator pubkey.

If not different, a switch to a compounding withdrawals credential can occur at the consensus layer (see `process_consolidation_request`: Electra Spec).

It is unclear if this is a desirable side-effect when invoking the `consolidate` function.

Recommendation: Document if the `consolidation` function has a valid use case of switching the withdrawals credential to compound rewards.

If a valid use case, for improved separation of concerns, consider having a separate function specifically for this use case, changing the `consolidation` implementation to only handle the consolidating of validators.

Auditor Suggestions

S1 Remove Unchecked Arithmetic in `for` Loops

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: f92f7e06d1801e6995eab7e7dab6fcf12cf992fb .

Description: The codebase contains instances of unchecked arithmetic, specifically for loop counter increments. Since Solidity version 0.8.22, the compiler automatically optimizes simple loop counter increments without requiring explicit `unchecked` blocks. As the project is using Solidity 0.8.29, these manual optimizations are redundant and add unnecessary complexity to the code.

Recommendation: Remove all instances of `unchecked { i++; }` or similar constructs in loops throughout the codebase. The Solidity 0.8.29 compiler will automatically apply these optimizations, resulting in cleaner code while maintaining the same gas efficiency.

S2 Initializers Are Callable in the Implementation Contract

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 3ccaf2c9e505ced10535470d0ebbd819b33e4330 .

Description: The protocol fails to disable initializers in the implementation contract storage. As a result, an adversary can call the contract `initializer` directly using the implementation contract address.

Recommendation: Consider adding a call to the `_disableInitializers()` function in the constructor of the implementation contract.

S3 Function `LibUint256.toLittleEndian64` Should Expect a Uint64 Input Value

Fixed

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `b8467e3bdc55cd72804be4276989694a5c75b650` .

File(s) affected: `src/libraries/LibUint256.sol`

Description: The `toLittleEndian64` function reverts for any input value greater than $2^{64} - 1$. Hence, its input parameter should be `uint64` , instead of `uint256` .

Recommendation: Update the input parameter of `toLittleEndian64()` to use the `uint64` type. For consistency, also update the `_depositAmount` parameter in the `_depositValidator()` function to `uint64` , aligning the types used across related logic.

S4 Duplicated Checks in `OrchestratorV1.initialize` Can Be Removed to Aid Code Clarity Fixed

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `634d3e64cf6408eb72a79a77ae15b9c9d7eb137c` .

File(s) affected: `src/OrchestratorV1.sol`

Description: In `OrchestratorV1.initialize` , the following checks occur:

```
if (owner == address(0) || tvsBeacon == address(0) || beneficiaryAddress == address(0)
    || configManagerAddress == address(0) || depositContractAddress == address(0)
) revert InvalidAddress();
```

However, apart from the check on `owner` , all other arguments are validated later in the initialization process:

- `tvsBeacon` is validated in `TvsBeaconAddress.set(tvsBeacon)` , as in #L36
- `beneficiaryAddress` , `configManagerAddress` , `depositContractAddress` are all verified in `_saveBaseConfig` , as in #L40

Recommendation: Simplify the logic validation inside `initialize` by only checking if `owner` is a null address.

S5 License Change of Cloned Code Fixed

✓ **Update**

Marked as "Fixed" by the client.

Addressed in: `698ddc70e4ae58d997e1e049eae17b35a08b2cd0` .

File(s) affected: `src/libraries/LibBytes.sol`

Description: The `LibBytes.sol` file seems to have been copied from an [open source library](#) , but had its license changed in the present audited code.

```
//SPDX-License-Identifier: Proprietary
pragma solidity 0.8.29;

/// @title Lib Bytes
/// @notice This library helps manipulating bytes
library LibBytes {
    ...
}
```

Recommendation: Switch back to the clone code to its original license: Unlicense.

S6 Missing Initializer Modifier in `TVSClone` Acknowledged

i **Update**

Marked as "Acknowledged" by the client.

The client provided the following explanation:

We will leave the TVS Clone initialize function as it contradicts a fix recommended by other auditors (<https://github.com/liquid-collective/tvs/commit/deef66b6ee62be69594c6d3547730b24e81a6cdd>).

File(s) affected: `src/TVSNonUpgradeable/TVSClone.sol`

Description: The `initialize()` function in `TVSClone` is missing the `initializer()` modifier. Although this doesn't introduce an immediate vulnerability, since `_setupSecurity()` still uses the modifier, it diverges from established best practices and could introduce risks if the contract's initialization logic changes in future updates.

Recommendation: Add the `initializer()` modifier directly to the `initialize()` function.

S7 Missing `nonReentrant` Guard

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `8b7bed68555a6e3988d13bd2d9a4bb5fc9f94d6b`.

File(s) affected: `src/components/TVS.sol`

Description: The `sweep()` function sends `amountToSweep` to the caller-provided `recipient` address. Since this might be a contract, it could try to maliciously re-enter the TVS contract.

Recommendation: We recommend adding the missing `nonReentrant` guard.

S8 Critical Role Transfer Not Following Two-Step Pattern

Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

We acknowledge the widespread adoption of the two-step pattern for critical role transfers. However, we have consciously chosen not to implement it for this feature in favour of code simplicity and reduced complexity.

Description: Both the TVS and Orchestrator contracts include ownership transfer mechanisms: `transferOwnership()` for TVS and `transferOrchestrator()` for the Orchestrator. The latter also assigns key roles defined in `_getOwnerRoles()`. If either function is called with an invalid or uncontrollable address, the system could be left without an active owner or admin. As a result, any functions restricted by `onlyOwner` or `onlyRole(admin)` would become permanently inaccessible.

Recommendation: For the TVS contract, consider using OpenZeppelin's `Ownable2Step` contract to adopt a two-step ownership pattern in which the new owner must accept their position before the transfer is complete. Consider adopting a similar two-step pattern for the orchestrator's ownership transfer.

Definitions

- High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- Informational** – The issue does not post an immediate risk, but is relevant to security best practices or Defence in Depth.
- Undetermined** – The impact of the issue is uncertain.
- Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.

- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

The team created 167 tests for the TVS codebase and 118 for the SMS system. The test suites on both repositories can be executed by running the following commands:

```
forge install
forge test
```

* TVS Repo:

Analysing contracts...

Running tests...

Ran 8 tests for test/TVSClone.t.sol:TVSCloneInitializationTest

[PASS] testConstructorWithZeroConsolidationContract() (gas: 39230)

[PASS] testConstructorWithZeroWithdrawalContract() (gas: 39228)

[PASS] testDeployWithValidArguments() (gas: 98582)

[PASS] testInitializerWithZeroAddressBeneficiary() (gas: 64442)

[PASS] testInitializerWithZeroAddressOwner() (gas: 77603)

[PASS] testWithNonTVSImplementation() (gas: 85837)

[PASS] testWithZeroAddressBeneficiary() (gas: 64440)

[PASS] testWithZeroAddressOwner() (gas: 38809)

Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 235.98ms (44.59ms CPU time)

Ran 9 tests for test/TVSUpgradeable.t.sol:TVSUpgradeableInitializationTest

[PASS] testConstructorWithZeroConsolidationContract() (gas: 42391)

[PASS] testConstructorWithZeroImmutableBeaconFactory() (gas: 40288)

[PASS] testConstructorWithZeroWithdrawalContract() (gas: 42389)

[PASS] testDeployWithValidArguments() (gas: 242248)

[PASS] testWithNonContractBeacon() (gas: 47596)

[PASS] testWithNonTVSImplementation() (gas: 304334)

[PASS] testWithZeroAddressBeacon() (gas: 43009)

[PASS] testWithZeroAddressBeneficiary() (gas: 75439)

[PASS] testWithZeroAddressOwner() (gas: 75396)

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 235.85ms (45.89ms CPU time)

Ran 32 tests for test/TVSClone.t.sol:TVSCloneTest

[PASS] testConsolidateEmitsEventWhenExcessRefundsFail() (gas: 98653)

[PASS] testConsolidateEmitsEventsForMultipleConsolidations() (gas: 72666)

[PASS] testConsolidateFailsIfFeeExceedsMax() (gas: 31598)

[PASS] testConsolidateFailsIfFeeExceedsValue() (gas: 43368)

[PASS] testConsolidateFailsIfNoValueSent() (gas: 22807)

[PASS] testConsolidateFailsIfNotOwner() (gas: 30496)

[PASS] testConsolidateFailsIfRequestFails() (gas: 41988)

[PASS] testConsolidateRefundsSenderAnyExcessFund() (gas: 77872)

[PASS] testConsolidateWorksIfAllIsFine() (gas: 50253)

[PASS] testNonOwnerCanSweepToDefaultBeneficiary() (gas: 58538)

[PASS] testRenounceOwnershipReverts() (gas: 16467)

[PASS] testSetBeneficiaryAsUnauthorized() (gas: 19513)

[PASS] testSetBeneficiaryWithInvalidAddress() (gas: 16971)

[PASS] testSetBeneficiaryWithValidAddress() (gas: 29587)

[PASS] testSweepAllWhenZeroBalance() (gas: 22026)

[PASS] testSweepToContractWithNoSweepToContractInterface() (gas: 17531)

[PASS] testSweepToContractWithSweepToContractInterfaceWorks() (gas: 98669)

[PASS] testSweepToCustomBeneficiaryFailsIfNotOwner() (gas: 18711)

[PASS] testSweepWithFailsWhenBalanceInsufficient() (gas: 16631)

[PASS] testTransfer() (gas: 39571)

[PASS] testTransferFailsIfNotOwner() (gas: 22031)

[PASS] testVersion() (gas: 13432)

[PASS] testWithdrawEmitsEventsForMultipleWithdrawals() (gas: 68345)

[PASS] testWithdrawFailsIfFeeExceedsMax() (gas: 32594)

[PASS] testWithdrawFailsIfFeeExceedsValue() (gas: 30026)

[PASS] testWithdrawFailsIfFeeReadFails() (gas: 31430)

[PASS] testWithdrawFailsIfInputLengthMismatch() (gas: 29065)

[PASS] testWithdrawFailsIfNotOwner() (gas: 31324)

```
[PASS] testWithdrawFailsIfRequestFails() (gas: 42242)
[PASS] testWithdrawRefundsSenderAnyExcessFund() (gas: 76670)
[PASS] testWithdrawWorksIfAllIsFine() (gas: 48621)
[PASS] testWithdrawmitsEventWhenExcessRefundsFail() (gas: 97427)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 254.56ms (66.08ms CPU time)
```

Ran 36 tests for test/TVSImmutable.t.sol:TVSImmutableTest

```
[PASS] testConsolidateEmitsEventWhenExcessRefundsFail() (gas: 95834)
[PASS] testConsolidateEmitsEventsForMultipleConsolidations() (gas: 69846)
[PASS] testConsolidateFailsIfFeeExceedsMax() (gas: 28882)
[PASS] testConsolidateFailsIfFeeExceedsValue() (gas: 40629)
[PASS] testConsolidateFailsIfNoValueSent() (gas: 20052)
[PASS] testConsolidateFailsIfNotOwner() (gas: 27716)
[PASS] testConsolidateFailsIfRequestFails() (gas: 39277)
[PASS] testConsolidateRefundsSenderAnyExcessFund() (gas: 75119)
[PASS] testConsolidateWorksIfAllIsFine() (gas: 47522)
[PASS] testConstructorWithZeroAddressBeneficiary() (gas: 92704)
[PASS] testConstructorWithZeroAddressOwner() (gas: 66941)
[PASS] testConstructorWithZeroConsolidationContract() (gas: 44588)
[PASS] testConstructorWithZeroWithdrawalContract() (gas: 44520)
[PASS] testNonOwnerCanSweepToDefaultBeneficiary() (gas: 55863)
[PASS] testRenounceOwnershipReverts() (gas: 13824)
[PASS] testSetBeneficiaryAsUnauthorized() (gas: 16816)
[PASS] testSetBeneficiaryWithInvalidAddress() (gas: 14344)
[PASS] testSetBeneficiaryWithValidAddress() (gas: 26773)
[PASS] testSweepAllWhenZeroBalance() (gas: 19395)
[PASS] testSweepToContractWithNoSweepToContractInterface() (gas: 14880)
[PASS] testSweepToContractWithSweepToContractInterfaceWorks() (gas: 95671)
[PASS] testSweepToCustomBeneficiaryFailsIfNotOwner() (gas: 15987)
[PASS] testSweepWithFailsWhenBalanceInsufficient() (gas: 14014)
[PASS] testTransfer() (gas: 36586)
[PASS] testTransferFailsIfNotOwner() (gas: 19395)
[PASS] testVersion() (gas: 10776)
[PASS] testWithdrawEmitsEventsForMultipleWithdrawals() (gas: 65498)
[PASS] testWithdrawFailsIfFeeExceedsMax() (gas: 29857)
[PASS] testWithdrawFailsIfFeeExceedsValue() (gas: 27312)
[PASS] testWithdrawFailsIfFeeReadFails() (gas: 28699)
[PASS] testWithdrawFailsIfInputLengthMismatch() (gas: 26334)
[PASS] testWithdrawFailsIfNotOwner() (gas: 28590)
[PASS] testWithdrawFailsIfRequestFails() (gas: 39489)
[PASS] testWithdrawRefundsSenderAnyExcessFund() (gas: 73963)
[PASS] testWithdrawWorksIfAllIsFine() (gas: 45937)
[PASS] testWithdrawmitsEventWhenExcessRefundsFail() (gas: 94698)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 254.90ms (180.82ms CPU time)
```

Ran 40 tests for test/TVSUpgradeable.t.sol:TVSUpgradeableTest

```
[PASS] testConsolidateEmitsEventWhenExcessRefundsFail() (gas: 106212)
[PASS] testConsolidateEmitsEventsForMultipleConsolidations() (gas: 80314)
[PASS] testConsolidateFailsIfFeeExceedsMax() (gas: 39246)
[PASS] testConsolidateFailsIfFeeExceedsValue() (gas: 51082)
[PASS] testConsolidateFailsIfNoValueSent() (gas: 30411)
[PASS] testConsolidateFailsIfNotOwner() (gas: 38077)
[PASS] testConsolidateFailsIfRequestFails() (gas: 49679)
[PASS] testConsolidateRefundsSenderAnyExcessFund() (gas: 85564)
[PASS] testConsolidateWorksIfAllIsFine() (gas: 57834)
[PASS] testImmutableBeaconDeploymentFailsWhenImplementationIsZero() (gas: 36482)
[PASS] testNonOwnerCanSweepToDefaultBeneficiary() (gas: 66103)
[PASS] testRenounceOwnershipReverts() (gas: 24170)
[PASS] testSetBeneficiaryAsUnauthorized() (gas: 27125)
[PASS] testSetBeneficiaryWithInvalidAddress() (gas: 24672)
[PASS] testSetBeneficiaryWithValidAddress() (gas: 38315)
[PASS] testSweepAllWhenZeroBalance() (gas: 29591)
[PASS] testSweepToContractWithNoSweepToContractInterface() (gas: 25229)
[PASS] testSweepToContractWithSweepToContractInterfaceWorks() (gas: 108566)
[PASS] testSweepToCustomBeneficiaryFailsIfNotOwner() (gas: 26254)
[PASS] testSweepWithFailsWhenBalanceInsufficient() (gas: 24241)
[PASS] testTransfer() (gas: 73210)
[PASS] testTransferFailsIfNotOwner() (gas: 29708)
[PASS] testTransferFailsWithInvalidBeacon() (gas: 2856965)
[PASS] testUpdateBeaconAsUnauthorized() (gas: 27115)
[PASS] testUpdateBeaconWithInvalidAddress() (gas: 24348)
[PASS] testUpdateUsingBeaconWithImplementationWithoutUnsafeSetBeaconFunction() (gas: 284219)
```



```
[PASS] testUpdateUsingSetBeaconWithoutImplementationFunction() (gas: 85686)
[PASS] testUpdateUsingSetBeaconFunction() (gas: 264732)
[PASS] testUpdateUsingSetBeaconUncheckedFunction() (gas: 261750)
[PASS] testVersion() (gas: 21109)
[PASS] testWithdrawEmitsEventsForMultipleWithdrawals() (gas: 75882)
[PASS] testWithdrawFailsIfFeeExceedsMax() (gas: 40176)
[PASS] testWithdrawFailsIfFeeExceedsValue() (gas: 37608)
[PASS] testWithdrawFailsIfFeeReadFails() (gas: 39012)
[PASS] testWithdrawFailsIfInputLengthMismatch() (gas: 36624)
[PASS] testWithdrawFailsIfNotOwner() (gas: 38839)
[PASS] testWithdrawFailsIfRequestFails() (gas: 49779)
[PASS] testWithdrawRefundsSenderAnyExcessFund() (gas: 84230)
[PASS] testWithdrawWorksIfAllIsFine() (gas: 56291)
[PASS] testWithdrawmitsEventWhenExcessRefundsFail() (gas: 104942)
Suite result: ok. 40 passed; 0 failed; 0 skipped; finished in 255.68ms (81.19ms CPU time)
```

```
Ran 42 tests for test/TVSFlexibleImmutable.t.sol:TVSFlexibleImmutableTest
[PASS] testConsolidateEmitsEventWhenExcessRefundsFail() (gas: 95856)
[PASS] testConsolidateEmitsEventsForMultipleConsolidations() (gas: 69891)
[PASS] testConsolidateFailsIfFeeExceedsMax() (gas: 28926)
[PASS] testConsolidateFailsIfFeeExceedsValue() (gas: 40696)
[PASS] testConsolidateFailsIfNoValueSent() (gas: 20052)
[PASS] testConsolidateFailsIfNotOwner() (gas: 27738)
[PASS] testConsolidateFailsIfRequestFails() (gas: 39277)
[PASS] testConsolidateRefundsSenderAnyExcessFund() (gas: 75208)
[PASS] testConsolidateWorksIfAllIsFine() (gas: 47456)
[PASS] testConstructorWithZeroAddressBeneficiary() (gas: 93156)
[PASS] testConstructorWithZeroAddressOwner() (gas: 67393)
[PASS] testConstructorWithZeroConsolidationContract() (gas: 44952)
[PASS] testConstructorWithZeroWithdrawalContract() (gas: 44994)
[PASS] testNonOwnerCanSweepToDefaultBeneficiary() (gas: 55841)
[PASS] testRenounceOwnershipReverts() (gas: 13869)
[PASS] testSetBeneficiaryAsUnauthorized() (gas: 16861)
[PASS] testSetBeneficiaryWithInvalidAddress() (gas: 14389)
[PASS] testSetBeneficiaryWithValidAddress() (gas: 26796)
[PASS] testSweepAllWhenZeroBalance() (gas: 19373)
[PASS] testSweepToContractWithNoSweepToContractInterface() (gas: 14924)
[PASS] testSweepToContractWithSweepToContractInterfaceWorks() (gas: 95673)
[PASS] testSweepToCustomBeneficiaryFailsIfNotOwner() (gas: 15965)
[PASS] testSweepWithFailsWhenBalanceInsufficient() (gas: 13970)
[PASS] testTransfer() (gas: 36498)
[PASS] testTransferFailsIfNotOwner() (gas: 19395)
[PASS] testVersion() (gas: 10754)
[PASS] testWithdrawEmitsEventsForMultipleWithdrawals() (gas: 65520)
[PASS] testWithdrawFailsIfFeeExceedsMax() (gas: 29857)
[PASS] testWithdrawFailsIfFeeExceedsValue() (gas: 27334)
[PASS] testWithdrawFailsIfFeeReadFails() (gas: 28699)
[PASS] testWithdrawFailsIfInputLengthMismatch() (gas: 26334)
[PASS] testWithdrawFailsIfNotOwner() (gas: 28590)
[PASS] testWithdrawFailsIfRequestFails() (gas: 39424)
[PASS] testWithdrawRefundsSenderAnyExcessFund() (gas: 73963)
[PASS] testWithdrawWorksIfAllIsFine() (gas: 45959)
[PASS] testWithdrawmitsEventWhenExcessRefundsFail() (gas: 94634)
[PASS] test_ExecuteBatch_RevertWhen_CallerNotOwner() (gas: 18911)
[PASS] test_ExecuteBatch_ShouldBeAtomic() (gas: 51277)
[PASS] test_ExecuteBatch_TargetCalledNTimes(uint256) (runs: 1000, μ: 157582, ~: 160698)
[PASS] test_ExecuteCallWhen_IsDelegateCallIsFalse() (gas: 78144)
[PASS] test_ExecuteDelegateCallWhen_IsDelegateCallIsTrue() (gas: 51784)
[PASS] test_RevertWhen_CallerNotOwner() (gas: 29449)
Suite result: ok. 42 passed; 0 failed; 0 skipped; finished in 1.44s (1.26s CPU time)
```

Ran 6 test suites in 1.50s (2.68s CPU time): 167 tests passed, 0 failed, 0 skipped (167 total tests)

* SMS Repo:

Analysing contracts...

Running tests...

```
Ran 2 tests for test/OrchestratorUpgradeable.t.sol:SettingOrchestratorAddressesTest
[PASS] test_setBeneficiary() (gas: 33008)
[PASS] test_setConfigmanagerAddress() (gas: 32967)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 8.45ms (348.29μs CPU time)
```



```
Ran 3 tests for test/OrchestratorUpgradeable.t.sol:RequestExitTest
[PASS] test_RequestExitsGivenAmountIsEqualTo0() (gas: 25295)
[PASS] test_RequestExitsGivenAmountIsGreaterThan0() (gas: 25274)
[PASS] test_RequestExitsWhenCalledByNon_WITHDRAWER_ROLE() (gas: 20240)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 8.55ms (935.88µs CPU time)

Ran 4 tests for test/OrchestratorUpgradeable.t.sol:RoleManagementTest
[PASS] test_RoleManagement_grantRolesGivenAValidAccountAndMultipleRoles() (gas: 127811)
[PASS] test_RoleManagement_grantRolesGivenAValidAccountAndSingleRole() (gas: 58741)
[PASS] test_RoleManagement_grantRolesGivenAnEmptyRolesArray() (gas: 40004)
[PASS] test_RoleManagement_grantRolesWhenCalledByNon_DEFAULT_ADMIN_ROLE() (gas: 31224)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.90ms (1.20ms CPU time)

Ran 3 tests for test/OrchestratorUpgradeable.t.sol:TransferOrchestratorTests
[PASS] test_TransferOrchestratorGivenNewOwnerIsEqualToAddressZero() (gas: 35202)
[PASS] test_TransferOrchestratorGivenValidNewOwnerAddress() (gas: 424811)
[PASS] test_TransferOrchestratorWhenCalledByNon_DEFAULT_ADMIN_ROLE() (gas: 45355)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 9.65ms (1.78ms CPU time)

Ran 4 tests for test/OrchestratorUpgradeable.t.sol:SetBeneficiaryTests
[PASS] test_SetBeneficiaryGivenEmptyValidatorSetsArray() (gas: 24166)
[PASS] test_SetBeneficiaryGivenValidValidatorSetsArrayAndBeneficiary() (gas: 117363)
[PASS] test_SetBeneficiaryGivenZeroAddress() (gas: 21836)
[PASS] test_SetBeneficiaryWhenCalledByNon_BENEFICIARY_ROLE() (gas: 87945)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 9.73ms (1.71ms CPU time)

Ran 2 tests for test/OrchestratorUpgradeable.t.sol:SweepTest
[PASS] test_SweepAllAvailableFundsInValidatorSets() (gas: 507110)
[PASS] test_SweepGivenAmountExceedsAvailableFundsInValidatorSet() (gas: 451296)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.89ms (722.92µs CPU time)

Ran 3 tests for test/OrchestratorUpgradeable.t.sol:UpgradeabilityTest
[PASS] test_UpgradeabilityGivenAValidNewImplementation() (gas: 67344)
[PASS] test_UpgradeabilityGivenAnInvalidNewImplementationForExampleANon_contractAddress() (gas: 61058)
[PASS] test_UpgradeabilityWhenUpgradedByNon_DEFAULT_ADMIN_ROLE() (gas: 56224)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.93ms (1.42ms CPU time)

Ran 5 tests for test/OrchestratorUpgradeable.t.sol:ValidatorSetTransferTest
[PASS] test_ValidatorSetTransferGivenEmptyValidatorSetsArray() (gas: 38630)
[PASS] test_ValidatorSetTransferGivenNewOwnerDoesNotHaveVALIDATOR_SET_RECIPIENT_ROLE() (gas: 474285)
[PASS] test_ValidatorSetTransferGivenValidValidatorSetsArrayAndNewBeneficiary() (gas: 770532)
[PASS] test_ValidatorSetTransferGivenValidValidatorSetsArrayAndNewBeneficiaryAndSweeps() (gas: 839101)
[PASS] test_ValidatorSetTransferWhenCalledByNon_VALIDATOR_SET_TRANSFER_ROLE() (gas: 475355)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.99ms (3.87ms CPU time)

Ran 3 tests for test/OrchestratorUpgradeable.t.sol:InitializationTest
[PASS] test_InitializeWhenInitializedWithAValidOwnerAndBeaconDepositContract() (gas: 658028)
[PASS] test_InitializeWhenInitializedWithAddressZeroAsOwner() (gas: 101677)
[PASS] test_InitializeWhenInitializedWithNonZeroStakerAndKeeperAddress() (gas: 841159)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 6.26ms (1.10ms CPU time)

Ran 4 tests for test/OrchestratorUpgradeable.t.sol:ConfigManagerTests
[PASS] test_ConfigManagerUpdatesGivenNewMinSweepAmountIsEqualTo0() (gas: 47455)
[PASS] test_ConfigManagerUpdatesGivenValidNewMinSweepAmount() (gas: 57684)
[PASS] test_ConfigManagerUpdatesWhenCalledByNon_CONFIG_MANAGER_ROLE() (gas: 65010)
[PASS] test_ReceiveFunctionWhenETHIsSentDirectlyToContract() (gas: 25243)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.59ms (714.96µs CPU time)

Ran 3 tests for test/OrchestratorUpgradeable.t.sol:DepositTest
[PASS] test_DepositGivenMsgValueIsEqualTo0() (gas: 23452)
[PASS] test_DepositGivenMsgValueIsGreaterThan0() (gas: 31094)
[PASS] test_DepositWhenCalledByNon_DEPOSITOR_ROLE() (gas: 28442)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.85ms (511.58µs CPU time)

Ran 4 tests for test/OrchestratorUpgradeable.t.sol:PausabilityTest
[PASS] test_AddressWithDEFAULT_ADMIN_ROLECanPause() (gas: 40296)
[PASS] test_OnlyAddressWithDEFAULT_ADMIN_ROLECanUnpause() (gas: 46144)
[PASS] test_OnlyAddressWithPAUSER_ROLECanPause() (gas: 69724)
[PASS] test_UnpauseUnCallableIfNotPaused() (gas: 21265)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.78ms (554.62µs CPU time)
```

```
Ran 3 tests for test/OrchestratorUpgradeable.t.sol:ValidatorSetCreationTest
[PASS] test_ValidatorSetCreationGivenCountIsEqualTo0() (gas: 28308)
[PASS] test_ValidatorSetCreationGivenCountIsGreaterThan0AndValidBeneficiary() (gas: 675041)
[PASS] test_ValidatorSetCreationWhenCalledByNon_VALIDATOR_SET_CREATOR_ROLE() (gas: 22487)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 13.42ms (905.29µs CPU time)

Ran 5 tests for test/OrchestratorUpgradeable.t.sol:WithdrawETHTest
[PASS] test_WithdrawETHGivenAmountIsEqualTo0() (gas: 33799)
[PASS] test_WithdrawETHGivenAmountIsGreaterThanContractBalance() (gas: 24712)
[PASS] test_WithdrawETHGivenSufficientContractBalanceAndDestinationIsEqualToAddressZero() (gas: 59364)
[PASS] test_WithdrawETHGivenSufficientContractBalanceAndValidDestination(uint256) (runs: 1500, µ: 61096, ~: 61561)
[PASS] test_WithdrawETHWhenCalledByNon_ETH_WITHDRAWER_ROLE() (gas: 21671)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 232.30ms (230.01ms CPU time)

Ran 12 tests for test/OrchestratorUpgradeable.t.sol:DepositValidatorSetTest
[PASS] testDepositFailsWithInvalidDepositRoot(bytes32) (runs: 1500, µ: 67323, ~: 67323)
[PASS] testDepositRevertsForInvalidPubKeyAndSignature() (gas: 330669)
[PASS] testDepositRevertsForUnequalPubKeyAndSignatureLength() (gas: 332014)
[PASS] testDepositValidatorSetToConsensusLayerFailsWithInvalidWC() (gas: 72982)
[PASS] testDepositValidatorSetToConsensusLayerFailsWithNoKeys() (gas: 265732)
[PASS] test_DepositValidatorSetToConsensusLayerFailsWhenOwnerOfValidatorSetIsNotOrchestrator() (gas: 460499)
[PASS] test_DepositValidatorSetToConsensusLayerGivenDepositRootDoesNotMatchDepositContract() (gas: 342920)
[PASS] test_DepositValidatorSetToConsensusLayerGivenValidValidatorSetsWithType0x01Credentials() (gas: 547519)
[PASS] test_DepositValidatorSetToConsensusLayerGivenValidValidatorSetsWithType0x02Credentials() (gas: 547489)
[PASS] test_DepositValidatorSetToConsensusLayerGivenValidatorSetsWithInvalidPublicKeyLength() (gas: 388627)
[PASS] test_DepositValidatorSetToConsensusLayerGivenWithdrawalContractAddressIsEqualToAddressZero() (gas: 103852)
[PASS] test_DepositValidatorSetToConsensusLayerWhenCalledByNon_KEEPER_ROLE() (gas: 318332)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 317.09ms (327.04ms CPU time)

Ran 8 tests for test/OrchestratorUpgradeable.t.sol:AccessControlOverrideTest
[PASS] test_DEFAULT_ADMIN_ROLE_CANNOT_GRANT_DEFAULT_ADMIN_ROLE() (gas: 36250)
[PASS] test_DEFAULT_ADMIN_ROLE_CANNOT_RENOUNCE_DEFAULT_ADMIN_ROLE() (gas: 22253)
[PASS] test_DEFAULT_ADMIN_ROLE_CANNOT_REVOKE_DEFAULT_ADMIN_ROLE() (gas: 36207)
[PASS] test_DEFAULT_ADMIN_ROLE_CAN_RENOUNCE() (gas: 27856)
[PASS] test_ONLY_DEFAULT_ADMIN_ROLE_CAN_GRANT_MULTIPLE_ROLES(address) (runs: 1500, µ: 249502, ~: 249502)
[PASS] test_ONLY_DEFAULT_ADMIN_ROLE_CAN_GRANT_ROLE(address) (runs: 1500, µ: 70133, ~: 70133)
[PASS] test_ONLY_DEFAULT_ADMIN_ROLE_CAN_REVOKE_MULTIPLE_ROLES(address) (runs: 1500, µ: 79304, ~: 79304)
[PASS] test_ONLY_DEFAULT_ADMIN_ROLE_CAN_REVOKE_ROLE(address) (runs: 1500, µ: 47669, ~: 47669)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 615.11ms (1.58s CPU time)

Ran 32 tests for test/ConfigManager.1.t.sol:ConfigManagerV1Test
[PASS] testAddBin() (gas: 606424)
[PASS] testAddBinFailsForUnsortedBin() (gas: 441327)
[PASS] testAddBinWithDuplicateNodeOperatorFailsWithInvalidNodeOperatorOrder() (gas: 441326)
[PASS] testCannotSetConfigManagerAsZeroAddress() (gas: 21954)
[PASS] testSetAllocationConfig() (gas: 469327)
[PASS] testSetAllocationConfigFailsWithInvalidConfig() (gas: 442754)
[PASS] testSetAllocationConfigFailsWithInvalidDecreaseInNumberOfBins() (gas: 441061)
[PASS] testSetAllocationConfigFailsWithUnauthorized() (gas: 441166)
[PASS] testSetAllocationConfigToUpdateExistingConfiguration() (gas: 563204)
[PASS] testSetAllocationConfigToUpdateExistingConfigurationSwappingOutNOsInBin() (gas: 562453)
[PASS] testSetAllocationConfigWithTenBins() (gas: 1173977)
[PASS] testTestSetAllocationConfigFailsInvalidDecreaseOfValidatorSetSize() (gas: 449457)
[PASS] testTestSetAllocationConfigFailsInvalidPerBinAllocation() (gas: 462737)
[PASS] testTestSetAllocationConfigFailsInvalidZeroValidatorSetSize() (gas: 449566)
[PASS] testTestSetAllocationConfigFailsWithInvalidDecraseInNOsPerBin() (gas: 454405)
[PASS] testTestSetAllocationConfigFailsWithInvalidDuplicateNodeOperator() (gas: 451770)
[PASS] testTestSetAllocationConfigFailsWithInvalidTotalAllocation() (gas: 445523)
[PASS] testUpdateBinWeightings() (gas: 444995)
[PASS] testUpdateBinWeightingsFailsWithInvalidSizeUpdate() (gas: 433822)
[PASS] testUpdateBinWeightingsFailsWithInvalidTotalAllocation() (gas: 435946)
[PASS] testUpdateBinWeightingsFailsWithUnauthorized() (gas: 506172)
[PASS] testUpdateBinWeightingsToZeroPercentBin() (gas: 445104)
[PASS] testUpdateNOWeightingsToZeroPercentNO() (gas: 439912)
[PASS] testUpdateNodeOperatorWeightings() (gas: 439826)
```

```
[PASS] testUpdateNodeOperatorWeightingsFailsWithInvalidPerBinAllocation() (gas: 437759)
[PASS] testUpdateNodeOperatorWeightingsFailsWithInvalidSizeUpdate() (gas: 434248)
[PASS] testUpdateNodeOperatorWeightingsFailsWithUnauthorized() (gas: 506254)
[PASS] testUpdateValidatorSetSizes() (gas: 441888)
[PASS] testUpdateValidatorSetSizesFailsWithInvalidDecreaseInValidatorSetSize() (gas: 435251)
[PASS] testUpdateValidatorSetSizesFailsWithInvalidSizeUpdate() (gas: 433842)
[PASS] testUpdateValidatorSetSizesFailsWithUnauthorized() (gas: 506241)
[PASS] testUpdateValidatorSetSizesFailsWithZeroValidatorSetSize() (gas: 434647)
Suite result: ok. 32 passed; 0 failed; 0 skipped; finished in 635.43ms (58.40ms CPU time)

Ran 2 tests for test/OrchestratorUpgradeable.t.sol:ConsolidationTest
[PASS] test_RequestGoesThroughIfFeeIsLessThanMaxFeePerConsolidation(uint256,uint256) (runs: 1500, μ: 322055, ~: 322055)
[PASS] test_revertIfFeeMoreThanMaxFeePerConsolidation(uint256,uint256) (runs: 1500, μ: 284059, ~: 284059)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 679.24ms (1.26s CPU time)

Ran 3 tests for test/OrchestratorUpgradeable.t.sol:WithdrawalTest
[PASS] test_RequestGoesThroughIfFeeIsLessThanMaxFeePerWithdrawal(uint256,uint256) (runs: 1500, μ: 300796, ~: 300908)
[PASS] test_RevertsDueToArrayLengthMismatch(uint256,uint256) (runs: 1500, μ: 259801, ~: 259801)
[PASS] test_revertIfFeeMoreThanMaxFeePerWithdrawal(uint256,uint256) (runs: 1500, μ: 276203, ~: 276427)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 765.37ms (1.55s CPU time)

Ran 11 tests for test/OrchestratorFactory.t.sol:OrchestratorFactoryTest
[PASS] test_DeployOrchestrator() (gas: 863650)
[PASS] test_GetCurrentInstanceID() (gas: 869343)
[PASS] test_Initialize() (gas: 31191)
[PASS] test_RevertDeployOrchestrator_InvalidAddress() (gas: 173206)
[PASS] test_RevertDeployOrchestrator_NotDeployer() (gas: 30860)
[PASS] test_RevertInitialize_ZeroAddress() (gas: 140816)
[PASS] test_RevertSetImplementation_NotAdmin() (gas: 6960387)
[PASS] test_RevertUpgradeTo_NotAdmin() (gas: 1968037)
[PASS] test_SetOrchestratorImplementation() (gas: 6970332)
[PASS] test_UpgradeTo() (gas: 1971281)
[PASS] test_Version() (gas: 15251)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 815.85ms (5.38ms CPU time)

Ran 1 test for test/OrchestratorUpgradeable.t.sol:PausabilityInvariantTest
[PASS] invariant_OnlyUnpauseFunctionCanUnpause() (runs: 0, calls: 0, reverts: 44)

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.93s (1.92s CPU time)

Ran 1 test for test/OrchestratorUpgradeable.t.sol:AccessControlInvariantTest
[PASS] invariant_OnlyTransferOrchestratorFunctionCanChangeDefaultAdminRole() (runs: 0, calls: 0, reverts: 7)

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.17s (2.16s CPU time)

Ran 22 test suites in 2.18s (8.25s CPU time): 118 tests passed, 0 failed, 0 skipped (118 total tests)
```

Code Coverage

The codebase has excellent test code coverage at around 90% for both repositories. To run them, execute the following commands:

```
forge install
forge coverage
```

- TVS Repo:

File	% Lines	% Statements	% Branches	% Funcs
src/TVSNonUpgradeable/TVSClone.sol	100.00% (3/3)	100.00% (2/2)	100.00% (0/0)	100.00% (1/1)
src/TVSNonUpgradeable/TVSFlexibleImmutable.sol	100.00% (11/11)	100.00% (12/12)	100.00% (2/2)	100.00% (3/3)

File	% Lines	% Statements	% Branches	% Funcs
src/TVSNonUpgradeable/TVSImmutable.sol	100.00% (3/3)	100.00% (2/2)	100.00% (0/0)	100.00% (1/1)
src/TVSNonUpgradeable/TVSImmutableBase.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
src/TVSUpgradeable/ImmutableBeacon.sol	100.00% (3/3)	100.00% (3/3)	100.00% (1/1)	100.00% (1/1)
src/TVSUpgradeable/ImmutableBeaconFactory.sol	100.00% (2/2)	100.00% (1/1)	100.00% (0/0)	100.00% (1/1)
src/TVSUpgradeable/TVSUpgradeable.sol	100.00% (23/23)	100.00% (23/23)	100.00% (1/1)	100.00% (8/8)
src/TVSUpgradeable/proxies/TVSBeaconProxy.sol	68.42% (13/19)	59.09% (13/22)	25.00% (1/4)	66.67% (2/3)
src/TVSUpgradeable/state/proxy/Beacon.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
src/components/BaseSecurity.sol	100.00% (5/5)	100.00% (3/3)	100.00% (0/0)	100.00% (2/2)
src/components/TVS.sol	100.00% (77/77)	100.00% (85/85)	100.00% (13/13)	100.00% (14/14)
src/state/Beneficiary.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
test/TVS.t.sol	100.00% (1/1)	100.00% (0/0)	100.00% (0/0)	100.00% (1/1)
test/TVSClone.t.sol	100.00% (5/5)	100.00% (4/4)	100.00% (0/0)	100.00% (1/1)
test/TVSFlexibleImmutable.t.sol	100.00% (6/6)	100.00% (4/4)	100.00% (0/0)	100.00% (2/2)
test/TVSUpgradeable.t.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
Total	96.55% (168/174)	94.67% (160/169)	85.71% (18/21)	97.83% (45/46)

- SMS Repo:

File	% Lines	% Statements	% Branches	% Funcs
src/OrchestratorBase.sol	97.97% (145/148)	98.36% (120/122)	92.31% (12/13)	97.62% (41/42)
src/OrchestratorFactory.sol	100.00% (27/27)	100.00% (22/22)	100.00% (0/0)	100.00% (8/8)
src/OrchestratorUpgradeableBase.sol	100.00% (1/1)	100.00% (0/0)	100.00% (0/0)	100.00% (1/1)
src/OrchestratorV1.sol	100.00% (12/12)	100.00% (18/18)	100.00% (1/1)	100.00% (2/2)
src/batchContracts/BatchDeposit.sol	100.00% (37/37)	100.00% (46/46)	100.00% (8/8)	100.00% (5/5)
src/batchContracts/BatchPectra.sol	100.00% (15/15)	100.00% (15/15)	100.00% (1/1)	100.00% (4/4)

File	% Lines	% Statements	% Branches	% Funcs
src/batchContracts/BatchSetTVSBeneficiary.sol	100.00% (5/5)	100.00% (5/5)	100.00% (0/0)	100.00% (2/2)
src/batchContracts/BatchSweep.sol	87.50% (7/8)	87.50% (7/8)	0.00% (0/1)	100.00% (2/2)
src/batchContracts/BatchTVSTransfer.sol	77.78% (7/9)	87.50% (7/8)	100.00% (1/1)	50.00% (1/2)
src/batchContracts/BatchUpgradeableTVSCreator.sol	90.00% (9/10)	93.75% (15/16)	0.00% (0/1)	100.00% (1/1)
src/batchContracts/state/DepositContractAddress.sol	83.33% (5/6)	80.00% (4/5)	0.00% (0/1)	100.00% (2/2)
src/components/ConfigManager.sol	97.73% (86/88)	97.87% (92/94)	88.24% (15/17)	100.00% (8/8)
src/components/state/configManager/AllocationConfig.sol	83.33% (40/48)	80.00% (40/50)	66.67% (4/6)	100.00% (4/4)
src/components/state/configManager/MinSweepAmount.sol	100.00% (4/4)	100.00% (3/3)	100.00% (0/0)	100.00% (2/2)
src/libraries/LibBytes.sol	72.73% (16/22)	74.07% (20/27)	0.00% (0/2)	100.00% (1/1)
src/libraries/LibUint256.sol	100.00% (20/20)	100.00% (19/19)	100.00% (0/0)	100.00% (1/1)
src/libraries/LibUnstructuredStorage.sol	100.00% (12/12)	100.00% (6/6)	100.00% (0/0)	100.00% (6/6)
src/state/orchestratorFactory/InstanceID.sol	100.00% (4/4)	100.00% (3/3)	100.00% (0/0)	100.00% (2/2)
src/state/orchestratorFactory/OrchestratorImplementationAddress.sol	100.00% (6/6)	100.00% (5/5)	100.00% (1/1)	100.00% (2/2)
src/state/orchestratorUpgradeable/BeneficiaryAddress.sol	100.00% (6/6)	100.00% (5/5)	100.00% (1/1)	100.00% (2/2)
src/state/orchestratorUpgradeable/ConfigManagerAddress.sol	100.00% (6/6)	100.00% (5/5)	100.00% (1/1)	100.00% (2/2)
src/state/orchestratorUpgradeable/TVSBeaconAddress.sol	83.33% (5/6)	80.00% (4/5)	0.00% (0/1)	100.00% (2/2)
test/OrchestratorUpgradeable.t.sol	100.00% (103/103)	100.00% (98/98)	100.00% (0/0)	100.00% (16/16)
test/mocks/MockBeaconDepositContract.sol	71.43% (10/14)	69.23% (9/13)	50.00% (1/2)	75.00% (3/4)
test/utils/BytesGenerator.sol	100.00% (8/8)	100.00% (8/8)	100.00% (2/2)	100.00% (1/1)
Total	95.36% (596/625)	95.05% (576/606)	80.00% (48/60)	97.58% (121/124)

Changelog

- 2025-05-08 - Initial report
- 2025-05-30 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over \$200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

